DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

(Artificial Intelligence & Machine Learning)

Academic Year:	2026-27	**Name of Student:**	
Semester:	V	**Student ID:**	
Class / Branch:	TE – CSE (AI&ML)	**Roll No:**	
Subject:	WMD Lab	**Date of Submission:**	

Experiment No. 06

Lab Outcome: Implement backend services in Kotlin with database integration and secure authentication.

Aim: Write Kotlin program to implement exception handling and use collection operations (map, filter, reduce).

Software used: Kotlin

Theory: -

1. What is a Program?

A program is a set of instructions given to a computer to perform a task. For example, suppose we want the computer to calculate 10 + 20:

```
fun main() {
    val result = 10 + 20
    println(result)
}
```

Output:

```
30
```

The computer follows the instructions from top to bottom.

Part A — Exception Handling

2. What is an Exception?

Before understanding exception handling, we need to understand what can go wrong in a program. Imagine using a calculator: entering 10 ÷ 2 gives 5, but entering 10 ÷ 0 is not allowed. Similarly, a

program can encounter situations that it cannot normally handle. Such an unexpected problem during program execution is called an exception.

> *Simple definition — an exception is an unexpected problem that occurs while a program is running and interrupts its normal execution.*

3. Real-Life Analogy for Exception

Imagine a student travelling to college. Normal situation:

```
Home → Bus Stop → Bus → College
```

But suddenly, the bus breaks down. The student cannot continue using the normal route and needs an alternative:

```
Bus breakdown
      ↓
Find another bus
      ↓
Continue journey
      ↓
Reach college
```

Exception handling works similarly:

```
Program running
      ↓
Unexpected problem
      ↓
Exception occurs
      ↓
Handle the problem
      ↓
Program can continue safely
```

4. Why Do We Need Exception Handling?

Consider this program:

```
fun main() {
    val a = 10
    val b = 0

    val result = a / b

    println(result)
}
```

The program tries to perform 10 / 0, which is invalid — it will generate an exception and terminate

instead of continuing normally. Exception handling allows us to say: "If this problem occurs, don't suddenly stop the whole program. Handle the problem properly."

5. Common Exceptions in Kotlin

Situation	Possible Exception
Dividing by zero	ArithmeticException
Converting invalid text to a number	NumberFormatException
Accessing an invalid array/list position	IndexOutOfBoundsException
Using an invalid operation	Other runtime exceptions

For example:

```
val number = "hello".toInt()
```

The text "hello" cannot be converted into an integer, so an exception can occur.

6. Kotlin Exception Handling Keywords

Kotlin mainly provides four keywords for exception handling: try, catch, finally, and throw. The most commonly used structure is:

```
try {
    // Code that may cause an exception
}
catch (e: Exception) {
    // Code to handle the exception
}
finally {
    // Code that executes at the end
}
```

try

The try block contains code that might produce an exception:

```
try {
    val result = 10 / 0
}
```

We are telling Kotlin: "Try executing this code. Something may go wrong."

catch

The catch block handles the exception if one occurs:

```
try {
    val result = 10 / 0
}
catch (e: ArithmeticException) {
    println("Cannot divide by zero")
}
```

If division by zero occurs, Kotlin moves to the catch block. Output:

```
Cannot divide by zero
```

e — the exception variable

In catch (e: ArithmeticException), e is a variable containing information about the exception — think of it as e = information about what went wrong. It can be printed directly:

```
catch (e: ArithmeticException) {
    println(e.message)
}
```

finally

The finally block contains code that should execute whether an exception occurs or not:

```
try {
    val result = 10 / 2
    println(result)
}
catch (e: ArithmeticException) {
    println("Error occurred")
}
finally {
    println("Program execution completed")
}
```

Output:

```
5
Program execution completed
```

The finally block executes even though there was no exception.

7. Complete Exception Handling Example

```
fun main() {

    try {
        val number1 = 10
        val number2 = 0
```

```
        val result = number1 / number2

        println("Result = $result")
    }
    catch (e: ArithmeticException) {
        println("Error: Cannot divide by zero")
    }
    finally {
        println("Execution completed")
    }
}
```

Output:

```
Error: Cannot divide by zero
Execution completed
```

8. Execution Flow

```
Program starts
      ↓
Enter try block
      ↓
   10 / 0
      ↓
Exception occurs
      ↓
Skip remaining try statements
      ↓
  Go to catch
      ↓
Print error message
      ↓
 Execute finally
      ↓
  Program ends
```

9. Exception Handling with User Input

A more practical example — asking the user to enter their age:

```
fun main() {

    print("Enter your age: ")

    try {
        val age = readln().toInt()
```

```
        println("Your age is $age")
    }
    catch (e: NumberFormatException) {
        println("Please enter a valid number")
    }
}
```

If the user enters 20, the output is "Your age is 20". If the user enters "twenty", the output is "Please enter a valid number" — this is much better than allowing the program to crash.

Part B — Collections and Collection Operations

10. What is a Collection?

Suppose we have marks of five students: 75, 82, 68, 90, 55. Instead of creating five separate variables (val mark1 = 75, val mark2 = 82, ...), we can store them together:

```
val marks = listOf(75, 82, 68, 90, 55)
```

This is called a collection.

11. What is a List?

A List is a collection that stores multiple values in an ordered manner:

```
val marks = listOf(75, 82, 68, 90, 55)
```

Think of it like a row of boxes, each with a position, starting from index 0:

```
+----+----+----+----+----+
| 75 | 82 | 68 | 90 | 55 |
+----+----+----+----+----+
  0    1    2    3    4
```

12. Accessing List Elements

```
fun main() {

    val marks = listOf(75, 82, 68, 90, 55)

    println(marks[0])
    println(marks[1])
    println(marks[3])
}
```

Output:

```
75
```

```
82
90
```

13. What are Collection Operations?

Kotlin provides many useful functions to work with collections, including map(), filter(), reduce(), forEach(), find(), count(), and sorted(). For this experiment, we concentrate on map(), filter(), and reduce().

14. map() Operation

map() is used when we want to transform every element of a collection. Think of a machine: Input → Transformation → Output. For example, 10 → ×2 → 20. Therefore, [10, 20, 30] becomes [20, 40, 60].

Syntax

```
collection.map {
    transformation
}
```

```
val numbers = listOf(10, 20, 30)

val doubled = numbers.map {
    it * 2
}
```

Here, numbers is the original collection, it is the current element, and it * 2 is the transformation. Result: [20, 40, 60].

Complete map() Program

```
fun main() {

    val numbers = listOf(10, 20, 30, 40, 50)

    val doubledNumbers = numbers.map {
        it * 2
    }

    println("Original numbers: $numbers")
    println("Doubled numbers: $doubledNumbers")
}
```

Output:

```
Original numbers: [10, 20, 30, 40, 50]
Doubled numbers: [20, 40, 60, 80, 100]
```

Understanding it

it represents one element at a time. For [10, 20, 30], Kotlin effectively performs 10×2=20, 20×2=40, 30×2=60, so [10, 20, 30] becomes [20, 40, 60].

map() Real-Life Example

A teacher has marks [50, 60, 70, 80] and wants to add 5 grace marks to every student:

```
val marks = listOf(50, 60, 70, 80)

val updatedMarks = marks.map {
    it + 5
}

println(updatedMarks)
```

Output: [55, 65, 75, 85].

Remember — map() = Transform every element.

15. filter() Operation

Suppose we have [45, 75, 32, 90, 60] and only want marks greater than or equal to 50. We could manually check every value, but Kotlin provides filter().

What does filter() do?

filter() selects elements that satisfy a condition. Think of a security gate:

```
Collection
    ↓
Check condition
    ↓
Keep matching elements
    ↓
Remove non-matching elements
```

For [45, 75, 32, 90, 60] with condition mark >= 50, the result is [75, 90, 60].

Syntax

```
collection.filter {
    condition
}
```

```
val marks = listOf(45, 75, 32, 90, 60)

val passedMarks = marks.filter {
    it >= 50
```

```
}
```

Complete filter() Program

```
fun main() {

    val marks = listOf(45, 75, 32, 90, 60)

    val passedMarks = marks.filter {
        it >= 50
    }

    println("All marks: $marks")
    println("Passed marks: $passedMarks")
}
```

Output:

```
All marks: [45, 75, 32, 90, 60]
Passed marks: [75, 90, 60]
```

Remember — filter() = Select elements based on a condition.

16. reduce() Operation

Suppose we have [10, 20, 30, 40] and want to calculate the total. We could write 10 + 20 + 30 + 40, but Kotlin provides reduce().

What does reduce() do?

reduce() combines all elements of a collection to produce one final value:

```
10 + 20 = 30
30 + 30 = 60
60 + 40 = 100
```

Therefore [10, 20, 30, 40] → reduce() → 100.

Syntax

```
collection.reduce {
    accumulator, element -> operation
}
```

```
val numbers = listOf(10, 20, 30, 40)

val total = numbers.reduce {
    total, number -> total + number
}
```

The first parameter represents the accumulated result, and the second represents the current element. A common convention is acc, value:

```
val total = numbers.reduce { acc, value ->
    acc + value
}
```

Complete reduce() Program

```
fun main() {

    val numbers = listOf(10, 20, 30, 40)

    val total = numbers.reduce { acc, value ->
        acc + value
    }

    println("Numbers: $numbers")
    println("Total: $total")
}
```

Output:

```
Numbers: [10, 20, 30, 40]
Total: 100
```

How reduce() Works Internally

For listOf(10, 20, 30, 40), Kotlin performs approximately:

```
Step 1: acc = 10, value = 20  →  10 + 20 = 30
Step 2: acc = 30, value = 30  →  30 + 30 = 60
Step 3: acc = 60, value = 40  →  60 + 40 = 100

Final result: 100
```

17. Difference Between map(), filter() and reduce()

Function	Purpose	Result
map()	Transform every element	New collection
filter()	Select matching elements	New collection
reduce()	Combine elements	Single value

Easy memory trick — map = CHANGE, filter = SELECT, reduce = COMBINE.

For example, starting from [10, 20, 30, 40, 50]: map gives [20, 40, 60, 80, 100]; filter (>25) gives [30, 40, 50]; reduce gives 150.

18. Combining map(), filter() and reduce()

The real power comes when we combine these operations. Suppose student marks are [45, 60, 75, 30, 90], and we want to select students who scored 50 or more, add 5 grace marks, and calculate the total:

```
fun main() {

    val marks = listOf(45, 60, 75, 30, 90)

    val result = marks
        .filter {
            it >= 50
        }
        .map {
            it + 5
        }
        .reduce { acc, value ->
            acc + value
        }

    println("Final total = $result")
}
```

Execution: filter() keeps marks ≥ 50 → [60, 75, 90]; map() adds 5 → [65, 80, 95]; reduce() adds them → 65 + 80 + 95 = 240.

19. Function Chaining

The code above is called function chaining — the output of one operation becomes the input of the next:

```
marks
  ↓
filter()
  ↓
map()
  ↓
reduce()
  ↓
final result
```

This is a very important feature of Kotlin collections.

Part C — Combined Practical Demonstration

20. Problem Statement

Write a Kotlin program to accept marks of students, handle invalid input using exception handling, identify passing marks, add grace marks to passing students, and calculate their total using collection operations.

21. Complete Program

```
fun main() {

    try {

        print("Enter number of students: ")
        val n = readln().toInt()

        val marks = mutableListOf<Int>()

        for (i in 1..n) {

            print("Enter marks of student $i: ")
            val mark = readln().toInt()

            marks.add(mark)
        }

        println("\nOriginal Marks: $marks")

        // Select passing marks
        val passedMarks = marks.filter {
            it >= 40
        }

        println("Passing Marks: $passedMarks")

        // Add 5 grace marks
        val updatedMarks = passedMarks.map {
            it + 5
        }

        println("Marks after grace: $updatedMarks")

        // Calculate total
        val total = updatedMarks.reduce { acc, value ->
            acc + value
        }

        println("Total of updated passing marks: $total")
    }
    catch (e: NumberFormatException) {
```

```
        println("Error: Please enter numbers only.")

    }
    finally {

        println("Program execution completed.")
    }
}
```

22. Sample Execution

Suppose the user enters:

```
Enter number of students: 5

Enter marks of student 1: 35
Enter marks of student 2: 50
Enter marks of student 3: 65
Enter marks of student 4: 30
Enter marks of student 5: 80
```

Original collection: [35, 50, 65, 30, 80]. Step 1 — filter() with it >= 40 gives [50, 65, 80]. Step 2 — map() with it + 5 gives [55, 70, 85]. Step 3 — reduce() gives 55 + 70 + 85 = 210.

Final output:

```
Original Marks: [35, 50, 65, 30, 80]
Passing Marks: [50, 65, 80]
Marks after grace: [55, 70, 85]
Total of updated passing marks: 210
Program execution completed.
```

23. What Happens If the User Enters Invalid Data?

Suppose the user enters "fifty" for a mark. The statement readln().toInt() tries to convert "fifty" into an integer, which is impossible, so Kotlin generates a NumberFormatException. The catch block handles it and prints "Error: Please enter numbers only." followed by "Program execution completed." — instead of the program terminating with an ugly error message, a meaningful message is shown to the user.

24. Why Use mutableListOf() Here?

mutableListOf<Int>() is used because marks are added one by one using marks.add(mark). A normal list created using listOf() is read-only with respect to its collection structure — for example, listOf(50, 60, 70) does not support marks.add(80). When a collection needs to be built dynamically, mutableListOf() is used instead.

25. Important Beginner Concept: val vs var

A beginner may ask: "val marks = mutableListOf<Int>() — if it is val, how can I add elements?" val means the variable cannot be reassigned to another collection, but the contents of a mutable collection can still be changed:

```
val marks = mutableListOf<Int>()

marks.add(50)
marks.add(60)
```

This is allowed. However, marks = mutableListOf(70, 80) is not allowed, because that would assign a completely different collection to a val.

26. Another Practical Example — Product Prices

An online store has prices [500, 1200, 800, 2500, 300]. We want to select products costing ₹500 or more, apply a 10% discount, and calculate the total price:

```
fun main() {

    val prices = listOf(500.0, 1200.0, 800.0, 2500.0, 300.0)

    val total = prices
        .filter {
            it >= 500
        }
        .map {
            it * 0.90
        }
        .reduce { acc, price ->
            acc + price
        }

    println("Total after discount = ₹$total")
}
```

Flow: [500, 1200, 800, 2500, 300] → filter() → [500, 1200, 800, 2500] → map() → [450, 1080, 720, 2250] → reduce() → 4500.

27. map() vs filter() vs reduce() — Key Differences

Using map() on listOf(10, 20, 30, 40) with it * 2 gives [20, 40, 60, 80] — the number of elements remains the same. Using filter() with it >= 30 gives [30, 40] — the number of elements can decrease. filter() returns a collection, while reduce() with acc + value returns a single value, e.g. 100.

28. Important Limitation of reduce()

reduce() should be used carefully with an empty collection — there is no first element from which

reduce() can begin, so reduce() can throw an exception for an empty collection.

For beginners, remember — use reduce() only when you know the collection contains at least one element.

29. reduce() vs fold() — Basic Introduction

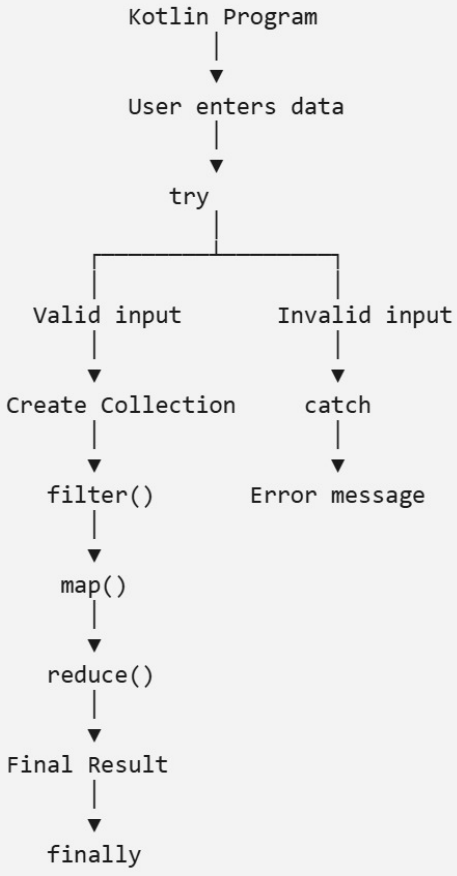
Kotlin also provides fold(), which is similar to reduce() but allows an initial value to be supplied:

```
val numbers = listOf(10, 20, 30)

val total = numbers.fold(0) { acc, value ->
    acc + value
}
```

Here the initial value is 0, then 0+10=10, 10+20=30, 30+30=60, giving a result of 60. For this experiment, the primary focus remains on map(), filter(), and reduce().

30. Exception Handling + Collections — Overall Flow





31. Key Theoretical Concepts to Remember

- **Exception —** an unexpected problem during program execution, e.g. 10 / 0.
- **Exception Handling —** a mechanism for handling runtime problems without abruptly terminating the program, using try / catch / finally.
- **Collection —** a structure used to store multiple values, e.g. val marks = listOf(50, 60, 70).
- **map() —** transforms every element (think: CHANGE).
- **filter() —** selects elements satisfying a condition (think: SELECT).
- **reduce() —** combines all elements into one result (think: COMBINE).

32. Quick Comparison Table

Concept	Purpose	Example	Output
try	Contains risky code	try { 10/0 }	—
catch	Handles exception	catch { }	Error message
finally	Executes at end	finally { }	Cleanup/final message
List	Stores multiple values	[10,20,30]	Collection
map()	Changes each element	it * 2	New collection
filter()	Selects elements	it > 20	Smaller collection
reduce()	Combines elements	acc + value	Single value

Implementation

Practical Experiment: AIML Club Event Registration& Revenue System

Problem Statement

Develop a Kotlin program for an AIML Club Event Management Portal to process entry fees collected from participants. The system receives raw string inputs (which may contain invalid entries), uses exception handling to safely convert valid numbers, and applies collection operations (filter, map, and reduce) to compute the final event collection.

Kotlin Implementation

D: > AY_2026-27 > ODD > WMD_VJshNair > Lab > Exp-6 > revenue.kt

```kotlin
fun main() {
    // Input list containing both valid fee amounts and invalid text entries
    val rawFeeInputs = listOf("500", "invalid_entry", "300", "800", "1200")
    val validFees = mutableListOf<Int>()

    println("--- Phase 1: Parsing Data with Exception Handling ---")
    for (input in rawFeeInputs) {
        try {
            // Converts string to integer; throws NumberFormatException if invalid
            val fee = input.toInt()
            validFees.add(fee)
        } catch (e: NumberFormatException) {
            println("Exception Handled: Unable to parse '$input'. Skipping entry.")
        }
    }

    println("\nParsed Valid Fees: $validFees")

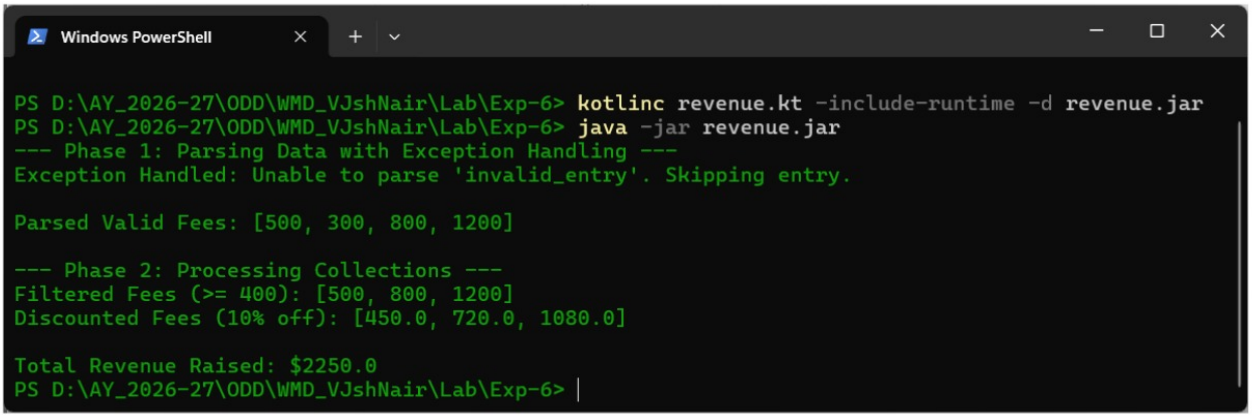
    println("\n--- Phase 2: Processing Collections ---")

    // 1. FILTER: Retain only standard/premium registration tiers (>= 400)
    val premiumFees = validFees.filter { fee -> fee >= 400 }
    println("Filtered Fees (>= 400): $premiumFees")

    // 2. MAP: Apply a 10% early-bird discount to each eligible fee
    val discountedFees = premiumFees.map { fee -> fee * 0.9 }
    println("Discounted Fees (10% off): $discountedFees")

    // 3. REDUCE: Aggregate all fees to calculate total collection
    val totalRevenue = discountedFees.reduce { total, fee -> total + fee }
    println("\nTotal Revenue Raised: $$totalRevenue")
}
```

Expected Output



Key Concepts Demonstrated

- **try-catch Block:** prevents program crashes when processing malformed user data, by catching NumberFormatException during runtime parsing.
- **filter:** scans the collection and extracts only items meeting a specified boolean condition (fee values ≥ 400).
- **map:** performs a transformation on every individual element of the collection (calculating 90% of the fee).
- **reduce:** accumulates values into a single summary output by combining collection elements sequentially from left to right.

This program simulates an automated event fee processor that converts messy user text inputs into clean numerical data, applies business rules, and calculates total revenue without crashing.

Data Transformation Breakdown

Stage	Input Data	Operation Applied	Resulting Output
1. Parsing	["500", "invalid_entry", "300", "800", "1200"]	Safely convert strings to integers using try-catch	[500, 300, 800, 1200]
2. Filter	[500, 300, 800, 1200]	Retain only fees >= 400	[500, 800, 1200]
3. Map	[500, 800, 1200]	Apply a 10% discount (fee * 0.9)	[450.0, 720.0, 1080.0]
4. Reduce	[450.0, 720.0, 1080.0]	Accumulate all values into a single sum	2250.0

Detailed Concept Breakdown

1. Exception Handling (try-catch)

- **The Problem:** in real-world applications, user inputs often contain typos or invalid text. Calling .toInt() on "invalid_entry" normally throws a NumberFormatException and crashes the program.
- **The Solution:** the try block attempts the conversion. If an error occurs, control immediately jumps to the catch block, which logs a message and keeps the program running smoothly.

2. Collection Operations

- **filter (Selection):** evaluates every element against a condition. It creates a new list containing only elements that return true (fee >= 400), safely dropping 300.
- **map (Transformation):** takes every item in the filtered list and transforms it based on a formula — here, multiplying each fee by 0.9 to compute a 10% early-bird discount.
- **reduce (Aggregation):** combines all list elements step-by-step into a single final value. It takes 450.0, adds 720.0 (equals 1170.0), then adds 1080.0 to yield 2250.0.

Out of 5 initial entries, the system successfully filters out broken data, processes the 3 eligible fee tiers, and computes a total revenue of $2250.0.

The primary goal of this program is to build a crash-resilient data processing pipeline that converts unpredictable raw inputs into clean, actionable business figures. In real-world software, data rarely arrives clean; this program demonstrates how to safely handle bad data, extract target entries, apply modifications in bulk, and aggregate results in four clear stages.

Stage 1 — Safe Data Parsing (Error Immunity)

Objective: prevent invalid user inputs from crashing the program. For example, if a user types "invalid_entry" or "N/A" into a registration form, standard number conversion ("N/A".toInt()) immediately crashes the application. Using try-catch allows the system to catch the error, log a warning, skip the bad text, and continue processing valid inputs like "500" → 500.

Stage 2 — Business Rule Filtering (Data Selection)

Objective: isolate only the entries that meet specific financial or operational criteria. For example, the event management team wants to process revenue from standard and VIP passes while excluding low-tier budget passes. Using .filter { fee >= 400 } automatically inspects [500, 300, 800, 1200], discards 300 for failing the threshold, and outputs [500, 800, 1200].

Stage 3 — Bulk Data Transformation (Mapping)

Objective: modify every remaining entry uniformly, without writing tedious manual for loops. Applying a 10% early-bird discount across all selected passes with .map { fee * 0.9 } recalculates every price simultaneously: 500×0.9=450.0, 800×0.9=720.0, 1200×0.9=1080.0, giving [450.0, 720.0, 1080.0].

Stage 4 — Revenue Aggregation (Reducing)

Objective: combine a list of processed values into a single final calculation. Using .reduce { total, fee -> total + fee } sequentially adds the values (450.0 + 720.0 + 1080.0) to produce one final output: $2250.0.

Suggested Practical Exercises

After completing the demonstration, students can try these independently.

Exercise 1 — Numbers

Create a list [10, 15, 20, 25, 30]. Use filter() to select numbers greater than 20, map() to multiply them by 3, and reduce() to calculate their total.

Exercise 2 — Student Marks

Create [35, 45, 60, 75, 90]. Filter marks ≥ 40, add 5 marks using map(), and calculate the total using reduce().

Exercise 3 — Product Prices

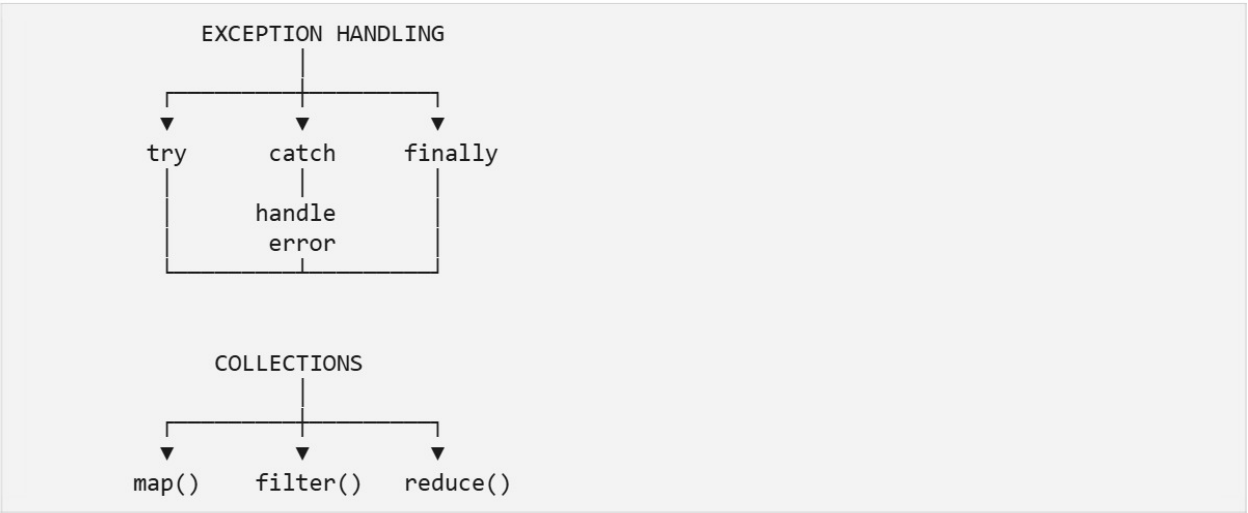
Create a list of five product prices. Filter products costing more than ₹500, apply a discount using map(), and calculate the total using reduce().

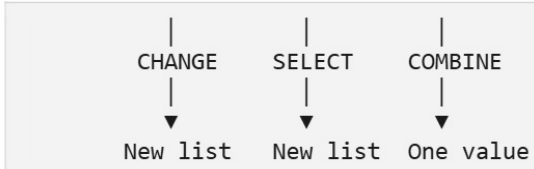
Exercise 4 — Exception Handling

Ask the user to enter two numbers and perform division (first number / second number). Use exception handling to display "Cannot divide by zero" when the second number is zero.

Final Takeaway

The most important concepts from this experiment can be remembered using this simple picture:





One-line memory trick — map() changes, filter() selects, and reduce() combines, while try-catch-finally helps the program handle errors safely.

Conclusion

Conclusion: -

Thus, we have demonstrated and implemented exception handling and used collection operations (map, filter, reduce) in Kotlin.